

Cardapiu Mobile

O **Cardapiu Mobile** é uma solução completa de delivery e menu digital projetada para aproximar clientes, restaurantes e entregadores de forma ágil e eficiente. Composto por um ecossistema robusto que integra um aplicativo móvel multiplataforma e uma API robusta, o sistema foi estruturado seguindo rigorosas práticas de arquitetura de software, garantindo escalabilidade, segurança e alta performance.

Nota do Projeto: Este repositório centraliza o desenvolvimento do ecossistema móvel, contendo tanto o código-fonte do backend em Java/Spring Boot quanto do frontend em Flutter.

1. Arquitetura Geral do Ecossistema

O projeto adota o modelo de arquitetura Cliente-Servidor distribuído, onde as responsabilidades são rigidamente separadas:

- **Back-End (API RESTful):** Desenvolvido em Java com o framework Spring Boot. Funciona como o núcleo inteligente do sistema, gerenciando persistência de dados, regras de negócio complexas, autenticação/autorização e envio de notificações em tempo real.
- **Front-End (Mobile):** Desenvolvido em Flutter (Dart), garantindo uma base de código única para Android e iOS com interface nativa de alta fidelidade e reatividade.

2. Funcionalidades Principais

Para Clientes

- Navegação fluida por catálogo de restaurantes e categorias.
- Visualização detalhada de produtos e composição de carrinho.
- Acompanhamento de pedidos em tempo real.
- Interface limpa e otimizada para conversão de vendas.

Para Entregadores

- Visualização e aceitação de entregas disponíveis.
- Atualização em tempo real do status de transporte.
- Compartilhamento contínuo de geolocalização.
- Painel de histórico de entregas finalizadas.

3. Tecnologias Utilizadas

Back-End

- **Java 17** & **Spring Boot 3.x** — Framework base da API.
- **Spring Security** & **JWT (JSON Web Tokens)** — Autenticação stateless e controle de perfis de acesso.
- **Spring Data JPA** — Abstração da camada de persistência de dados.
- **Firebase Admin SDK** — Integração nativa para notificações *push* e serviços em tempo real.

Front-End

- **Flutter** & **Dart** — Framework de desenvolvimento UI multiplataforma.
- **Riverpod / Provider** — Gerenciamento de estado previsível e injeção de dependências.
- **Dio / Http** — Cliente HTTP otimizado para comunicação assíncrona com a API.

4. Estrutura de Diretórios

```
cardapiu_mobile/
├── back-end/                                # Módulo do Servidor (Spring Boot)
│   ├── src/main/java/com/cardapiu/demo/
│   │   ├── controllers/                    # Exposição dos endpoints REST
│   │   ├── dtos/                          # Objetos de Transferência de Dados (Requests/Responses)
│   │   ├── exceptions/                    # Manipulação global de erros da API
│   │   ├── infra/                        # Configurações de Segurança (JWT) e Firebase
│   │   ├── models/                       # Entidades de domínio (Pedido, Usuário, Produto, etc.)
│   │   ├── repositories/                 # Interfaces de comunicação com o Banco de Dados
│   │   └── services/                     # Camada de lógica de negócio centralizada
│   └── pom.xml                             # Gerenciador de dependências Maven
├── front-end/                              # Módulo do Aplicativo (Flutter)
│   ├── lib/
│   │   ├── aplicativo/                   # Inicialização globais e providers do app
│   │   ├── compartilhado/               # Componentes visuais utilitários reutilizáveis
│   │   ├── funcionalidades/             # Módulos isolados (Autenticação, Pedidos, Rastreamento)
│   │   └── nucleo/                       # Configurações de rede, temas e utilitários de formatação
│   └── pubspec.yaml                       # Gerenciador de dependências do Flutter
```

5. Como Executar o Projeto

Pré-requisitos

Certifique-se de possuir instalados em sua máquina de desenvolvimento:

- Java JDK 17 ou superior.

- Apache Maven.
- Flutter SDK atualizado estável.
- Um banco de dados configurado (conforme suas propriedades do Spring).

Passo 1: Configuração e Execução do Back-End

Navegue até o diretório do servidor e configure as variáveis de ambiente:

```
cd back-end  
cp .env.example .env
```

Preencha o arquivo `.env` criado com as credenciais do seu banco de dados e chave secreta para a assinatura de tokens JWT. Em seguida, compile e execute a aplicação:

```
mvn clean install  
mvn spring-boot:run
```

Passo 2: Execução do Front-End (App Mobile)

Navegue até o diretório do cliente, instale as dependências necessárias e inicialize o emulador ou dispositivo físico:

```
cd ../front-end  
flutter pub get  
flutter run
```

6. Segurança e Perfis de Acesso

A API implementa proteção rigorosa de rotas baseada em papéis (*Role-Based Access Control - RBAC*). Os usuários são categorizados através do enumerador `UserRole`, garantindo que endpoints sensíveis (como atualização de status de entrega ou gerenciamento de cardápios) estejam protegidos contra acessos não autorizados por meio da validação criptográfica de tokens JWT em cada requisição.